

WEEK – 4(Skill)

Name: S Rikhil

Rollno: 2520030480

Section: 8

Experiment No. 7(Q7)

Title: Process Synchronization and Child Process Monitoring Using waitpid()

Aim: To implement process synchronization and child process monitoring using waitpid(). To retrieve the PATH variable, parse search directories, locate executables, verify permissions, handle missing commands, and test command resolution.

Objective:

Part 1 – Process Synchronization and Child Process Monitoring

- To create a child process using fork().
- To execute a child process.
- To monitor the child process using waitpid().
- To retrieve the child's termination status.
- To synchronize parent and child processes.
- To test child-process monitoring.

Part 2 – PATH Command Resolution

- To retrieve the PATH environment variable.
- To parse the directories present in PATH.
- To locate executable files.
- To verify executable permissions.
- To handle missing commands.
- To test command-resolution logic.

Theory:

1. Process Synchronization

Process synchronization controls the execution of related processes. When a parent process creates a child using fork(), both processes can execute independently. The parent can use waitpid() to wait for the child process to complete.

2. fork()

The fork() system call creates a new child process.

- It returns 0 to the child process.
- It returns the child's process ID to the parent.
- It returns -1 if process creation fails.

Example:

```
pid_t pid = fork();
```

3. waitpid()

waitpid() allows the parent process to wait for a specific child process.

```
waitpid(pid, &status, 0);
```

It ensures that the parent can monitor the child and obtain its termination status.

4. Child Process Monitoring

After waitpid() returns, the parent can check whether the child terminated normally using WIFEXITED() and obtain its exit status using WEXITSTATUS().

Example:

```
if (WIFEXITED(status))
    printf("Child exit status: %d", WEXITSTATUS(status));
```

PATH Command Resolution

5. PATH Environment Variable

PATH is an environment variable containing a list of directories where executable programs are located.

Example:

```
/usr/local/bin:/usr/bin:/bin
```

When a command is entered without an absolute path, the shell searches these directories to locate the executable.

6. Parse Search Directories

The PATH variable is separated using the colon : delimiter.

For example:

```
/usr/local/bin:/usr/bin:/bin
```

is divided into:

```
/usr/local/bin
```

/usr/bin

/bin

7. Locate Executables

For every directory in PATH, the command name is appended to the directory.

For example, for the command:

ls

the parser checks paths such as:

/usr/local/bin/ls

/usr/bin/ls

/bin/ls

8. Verify Permissions

The access() system call with X_OK is used to check whether the file exists and has executable permission.

```
if (access(candidate, X_OK) == 0)
```

If the condition is true, the executable has been located.

9. Missing Commands

If the command is not found in any PATH directory, the program displays:

Command not found

This prevents the program from failing when the user enters an invalid command.

Algorithm:

Part 1 – Process Synchronization

1. Start the program.
2. Create a child process using fork().
3. Check whether fork() was successful.
4. Execute the child process.
5. Display the child PID and parent PID.
6. Perform the child task.
7. Terminate the child with an exit status.

8. Parent process calls waitpid().
9. Wait for the child to terminate.
10. Check the termination status using WIFEXITED().
11. Obtain the exit status using WEXITSTATUS().
12. Display the result.
13. Stop the program.

Part 2 – PATH Command Resolution

1. Start the program.
2. Read the command from the user.
3. Retrieve the PATH variable using getenv().
4. Parse PATH using : as the delimiter.
5. Select each directory one by one.
6. Construct the complete executable path.
7. Check executable permission using access().
8. If the executable is found, display its path.
9. If the executable is not found in any directory, display an error.
10. Test valid and invalid commands.
11. Stop the program.

Functions Used:

Function	Purpose
main()	Starting point of the program
fork()	Creates a child process
waitpid()	Waits for and monitors a child process
getpid()	Retrieves the current process ID
getppid()	Retrieves the parent process ID
_exit()	Terminates the child process
WIFEXITED()	Checks whether the child exited normally
WEXITSTATUS()	Retrieves the child's exit status
getenv()	Retrieves the PATH variable
strtok()	Splits PATH into directories

Function	Purpose
access()	Checks executable permission
snprintf()	Creates the complete executable path
fgets()	Reads command input
printf()	Displays output and messages

Hardware and Operating System Relationship

Hardware consists of physical components such as CPU, RAM, storage devices, keyboard, mouse and display. The Operating System acts as an interface between application software and hardware. It manages processes, memory, files, devices and system resources.

Relationship

USER



APPLICATION SOFTWARE / SHELL



SYSTEM CALLS



OPERATING SYSTEM



STORAGE DEVICES

How OS interacts with hardware

1. **CPU Management** – OS schedules parent and child processes.
2. **Memory Management** – OS manages memory allocated to processes.
3. **File Management** – OS manages executable files and directories.
4. **Device Management** – OS controls input and output devices.
5. **Process Management** – OS creates, executes and terminates processes.

6. **Security** – OS controls access permissions to files and resources.

Observation

The program successfully creates a child process using `fork()` and synchronizes the parent process using `waitpid()`. The parent process waits for the child to terminate and retrieves its exit status using `WIFEXITED()` and `WEXITSTATUS()`.

The PATH resolver successfully retrieves the PATH environment variable, parses its directories, constructs executable paths, verifies executable permissions using `access()`, and handles missing commands.

Result

The process synchronization and child-process monitoring functionality using `waitpid()` was successfully implemented. The PATH command-resolution functionality was also successfully implemented to retrieve PATH, parse search directories, locate executables, verify permissions, handle missing commands, and test command resolution.

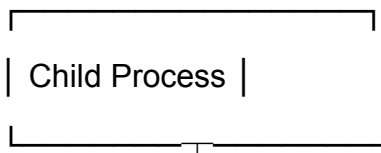
. **Design / Flow**

Process Synchronization Flow

Parent Process



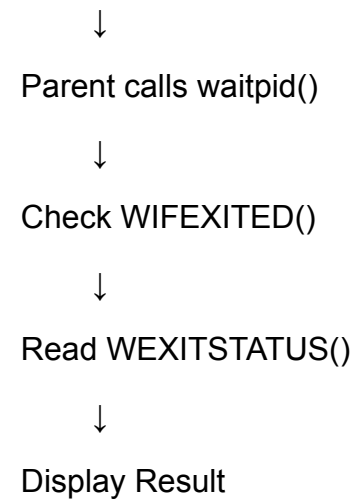
`fork()`



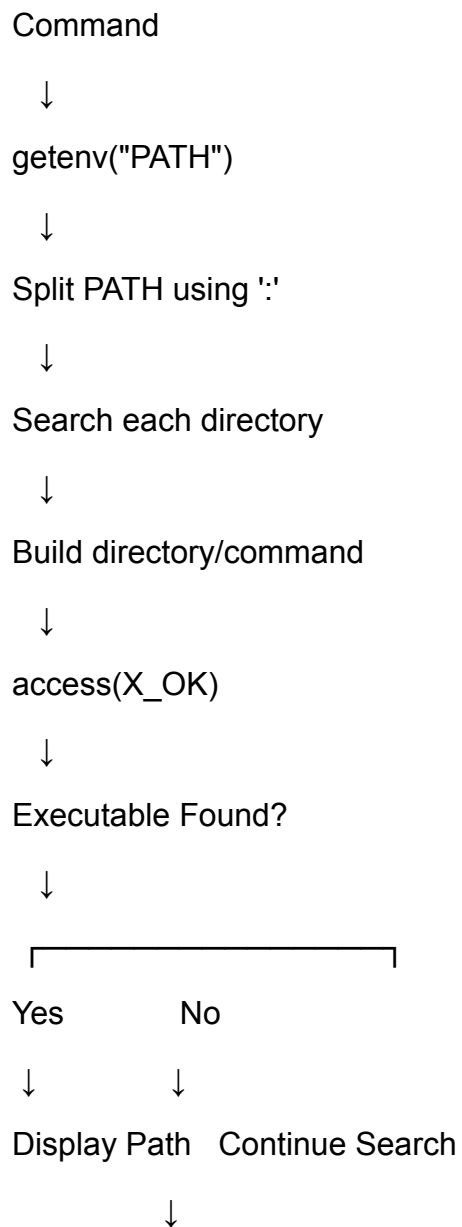
Perform Task



`exit(7)`



PATH Resolution Flow



No Directory Left



Command Not Found

CODE-

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
#define MAX 200
```

```
void resolveCommand(const char *cmd)
```

```
{
```

```
    char *path = getenv("PATH");
```

```
    if (strchr(cmd, '/') != NULL)
```

```
    {
```

```
        if (access(cmd, X_OK) == 0)
```

```
            printf("Executable found: %s\n", cmd);
```

```
        else
```

```
            printf("Command not executable or not found: %s\n", cmd);
```

```
        return;
```

```
    }
```

```
    if (path == NULL)
```

```

{
    printf("PATH variable is not set.\n");
    return;
}

char *copy = strdup(path);
char *dir = strtok(copy, ":");
char candidate[MAX];

while (dir != NULL)
{
    snprintf(candidate, sizeof(candidate),
              "%s/%s", dir, cmd);

    if (access(candidate, X_OK) == 0)
    {
        printf("Executable found: %s\n", candidate);
        free(copy);
        return;
    }

    dir = strtok(NULL, ":");
}

printf("Command not found: %s\n", cmd);

free(copy);
}

```

```

void monitorChild()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        printf("Child PID : %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("Child is performing its task...\n");

        _exit(7);
    }

    int status;

    printf("Parent created child: %d\n", pid);
    printf("Parent waiting using waitpid()...\n");

    if (waitpid(pid, &status, 0) == -1)
    {

```

```
    perror("waitpid");  
    return;  
}
```

```
if (WIFEXITED(status))  
{  
    printf("Child exited normally.\n");  
    printf("Child exit status: %d\n",  
        WEXITSTATUS(status));  
}  
}
```

```
int main()  
{  
    char command[100];  
  
    printf("=== Process Synchronization ===\n");  
  
    monitorChild();  
  
    printf("\n=== PATH Command Resolution ===\n");  
  
    printf("Enter command to locate: ");  
  
    if (fgets(command, sizeof(command), stdin) == NULL)  
        return 0;  
  
    command[strcspn(command, "\n")] = '\0';
```

```
if (command[0] == '\0')
{
    printf("No command entered.\n");
    return 0;
}

resolveCommand(command);

return 0;
}
```

Compilation and Execution

1. Compile

```
gcc question7.c -o question7
```

2. Execute

```
./question7
```

Sample Output

Process Synchronization Test

```
=== Process Synchronization ===
```

```
Parent created child: 4210
```

```
Parent waiting using waitpid()...
```

```
Child PID : 4210
```

```
Parent PID: 4209
```

```
Child is performing its task...
```

```
Child exited normally.
```

```
Child exit status: 7
```

PATH Resolution Test

```
=== PATH Command Resolution ===
```

```
Enter command to locate: ls
```

```
Executable found: /usr/bin/l
```

Valid Command

Enter command to locate: pwd

Executable found: /usr/bin/pwd

GCC Test

Enter command to locate: gcc

Executable found: /usr/bin/gcc

Missing Command

Enter command to locate: unknowncommand

Command not found: unknowncommand

Direct Path Test

Enter command to locate: /bin/lS

Executable found: /bin/lS

Conclusion

This experiment demonstrates process synchronization and child-process monitoring using `fork()` and `waitpid()`. The parent process creates a child, waits for its completion and retrieves its termination status.

The experiment also demonstrates PATH command resolution. The program retrieves the PATH variable, parses search directories, constructs possible executable paths, verifies executable permissions and handles missing commands. Thus, the experiment demonstrates important Operating System concepts such as process management, synchronization, environment variables, file permissions and command resolution.

OUTPUT-

```

teja@Hasini:~$ nano question7.c
teja@Hasini:~$ ls
Experiment-2      fork.c      os_task     path_resolver  quotes.c
filecopy.c.save  hello      os_task.c   path_resolver.c
filecopy.c.save.1 hello.c     parser      question7.c
fork             linux      parser.c    quotes
teja@Hasini:~$ gcc question7.c -o question7
teja@Hasini:~$ ls
Experiment-2      fork.c      os_task     path_resolver  quotes
filecopy.c.save  hello      os_task.c   path_resolver.c
filecopy.c.save.1 hello.c     parser      question7
fork             linux      parser.c    question7.c
teja@Hasini:~$
teja@Hasini:~$ ./question7
=== Process Synchronization ===
Parent created child: 533
Parent waiting using waitpid()...
Child PID : 533
Parent PID: 532
Child is performing its task...
Child exited normally.
Child exit status: 7

=== PATH Command Resolution ===
Enter command to locate: ls
Executable found: /usr/bin/ls
teja@Hasini:~$ ./question7
=== Process Synchronization ===
Parent created child: 544
Parent waiting using waitpid()...
Child PID : 544
Parent PID: 543
Child is performing its task...
Child exited normally.
Child exit status: 7

=== PATH Command Resolution ===

```

```

Child exit status: 7

```

```

=== PATH Command Resolution ===
Enter command to locate:
No command entered.
teja@Hasini:~$ pwd
/home/teja

```